Page No.:
DATE: 8/9/25

TCS - UT 1 @B

1). Explain with eg the Chomsky Heirarchy.

2). Let G = (V, T, P, S) be the CFG having foll set of ~~prblm~~ productions Derive the string "aabbaa" using Leftmost derivation and rightmost derivation.
S → aAS | a, A → SbA | SS | ba.

3). What is finite Automation? Give the finite automation Maccepting (a|b)* (baaa).

4). Describe finite state machine or DFA.

5) Give deterministic finite automata accepting the following languages over the alphabet {0,1}.
(a) No. of 1's is even & No. of 0's is even.
(b) No. of 1's is Odd & No. of 0's is odd.

6). Convert the foll. grammar into finite automata
S = aX | bY | a | b , X = aS | bY | b , Y = aX | bS

7). Write a shortnote on NFA.

8) Convert to DFA the foll. NFA.

	0	1
→ p	{p, q}	{p}
q	{r}	{r}
r	{s}	∅
s*	{s}	{s}

9) Construct an NFA an the Equivalent DFA accepting string over (0,1). whose every block of 4 consecutive symbols, contains atleast 3 errors.

10). Give the difference betn NFA & DFA.

11). Differentiate betn moore & mealy machine.

TCS:- UTI QB Answers.

Q.17 Explain with example the Chomsky Hierarchy.

Ans
- The Chomsky Hierarchy represents the class of languages that are accepted by different machines introduced by Naom Chomsky in 1956.
- It provides the framework for understanding the limitations and capabilities of different grammar & formal language.
- A grammar can be classified on the basis of production rules. Chomsky classified the grammar into the foll. 4 levels.

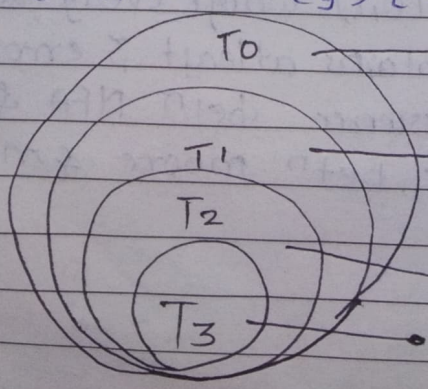
- Grammar → Language → m/c
  (generated by) (accepted/recognized by)

T-0 → recursive enumerable → Turing machine language
eg: set of all valid programs that halt

T-1 → Context-sens. Language → Linear bound Automata
from $\alpha \to \beta$ with $|\alpha| \leq |\beta|$ or β is strong as α
$L = \{a^n b^n c^n \mid n \geq 1\}$

T-2 → Context free Language → Pushdown Automata
from $A \to \alpha$ eg: $L = \{a^n b^n \mid n \geq 0\}$

T-3 → Regular Language → Finite Automata
from $A \to aB$ or $A \to a$ eg: $L = \{a^n b^m \mid n, m \geq 0\}$

- 

1] Type 0:- Unrestricted grammar
- generated by recurisively ennumerable languages where there are no production rules
- automation accepted or recognized by: Turing machine.
- eg:- $\{a^n b^n c^n \mid n \geq 1\}$
- keypoint: recognizable but not neccessarily decidable.

2] Type 1:- Context-Sensitive Grammar
- generated by context sensitive language where production are of from
  $\alpha \rightarrow \beta$, with $|\alpha| \leq |\beta|$
- automation by: Linear bound automata
- eg:- $\{a^n b^n c^n \mid n \geq 1\}$ can be generated with extra constraints.
- keypoint: useful language where structure depends on context

3] Type 2:- Context free ~~languages~~ grammar
- generated by Context free language where production are of form $A \rightarrow \alpha$
- automation: pushdown automata. (PDA)
- eg:- $\{a^n b^n \mid n \geq 0\}$
- keypoint: used in programming languages and compilers

4] Type 3: Regular ~~Languages~~ grammar
- generated by regular grammars where production are of form $A \to \alpha B$ or $A \to a$
- automation : finite automata
- eg: $\{a^n b^m | n, m > 0\}$
- keypoint: Simplest language used in pattern making.

Q. 2] Let $G = (V, T, P, S)$ be the CFG having foll. set of productions. Derive string "aabbaa" using Leftmost derivation & rightmost derivation.

→ $S \to aAS | a$, $A \to SbA | SS | ba$.

→

Leftmost derivation using parse tree	Rightmost derivation using parse tree.

→ S → aAS
S → a
A → SbA
A → SS
A → ba

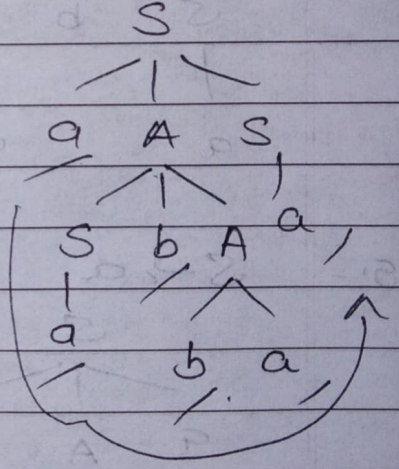
String : aabbaa

Let G = (V, T, P, S) be given grammar.

V = { A, S }
T = { a, b }
P = { S → aAS | a
A → SbA | SS | ba
S = start symbol

LMD : aabbaa
1 3 2 4 5 6

S → aAS (S → aAS)
S → aSbAS (A → SbA)
S → aabAS (S → a)
S → aabbaS (A → ba)
S → aabbaa (S → a)



DMD : aabbaa
1 6 3 4 5 2

S → aAS (S → aAS)
S → aAa (S → a)
S → aSbAa (A → SbA)
S → aSbbaa (A → ba)
S → aabbaa (S → a)

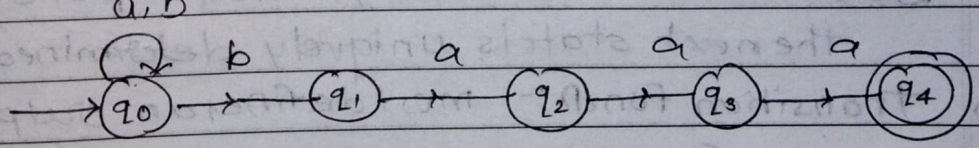
Q.3] What is finite Automation? Give the finite automation M accepting (a,b)*(baaa).

→ • finite Automation

- Finite automation can be thought as a severely restricted model of a computer.
- Every Computer has CPU ; it executes a prog. stored in a memory.
- This prog. normally accepts some input and delievers the processed result.
- A system where energy and information are transformed and used for performing some func^n s without direct involvement of men is called automation.
- A finite automata a.k.a. finite state machine.

• M accepting (a,b)*(baaa)

- RE = (a,b)*(baaa) represents a strings ending in baaa
- FA given is



Page No.:
DATE : / /

Q.4] Describe finite State Machine or DFA.

→

- A DFA (Deterministic finite Automata) is a mathematical machine that reads inputs string over a given alphabet and accepts or rejects them based on its structure.
- It is used to recognize regular language.
- A deterministic finite automata is a quintuple.

$M = (Q, \Sigma, \delta, q_0, F)$,

where,

Q = finite set of states

Σ = finite set of alphabets

&

δ = transition function $\delta : Q \times \Sigma \to Q$.

q_0 = Initial state

F = Final state / accepting state.

$F \subseteq Q$ [F is a subset of Q]

- Next state depends on current state & current input.
- key properties :-

1) Deterministic : For every state & input symbol the next state is uniquely determined

2) Transition func^n :- must define an output state for every pair.

3) Accepts strings :- it only ina as.

Page No.:
DATE : / /

Q.10] Give the difference betn NFA & DFA.

→

Parameter	NFA	DFA
Transition	Non deterministic	Deterministic
Power	NFA is as powerful as DFA.	DFA is as powerful as an NFA.
Design	Easy to design due to non-deterministic transition.	More difficult to design as transitions are deterministic
	All NFA are not DFA	All DFA are NFA.
	NFA can use empty string transition	DFA cannot use empty string transition.
	It requires less space.	It requires more space
	Backtracking is not possible	Possible
	Dead state is not required.	may be req.
	NFA can be understood as Multiple little machines Computing at a same time.	DFA can be understood at one machine.
	Next state is not determined	determined.
	Input can guess	Cannot guess

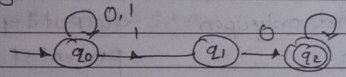
Q.11] Differentiate both Moore & Mealy Machine.

→

Moore Machine	Mealy Machine
Output is associated with state.	Output is associated with ~~transmission~~ transition
A moore machine has often a Larger number of states compare to the corresponding Mealy machine.	A mealy machine can be implemented with fewer no. of states.
Output depends only upon present state.	Output depends ~~only~~ on the present state as well as present input.
React slower to input	React faster to inputs
Easy to design	Difficult to design
Synchronous o/p & state generation	Asynchronous o/p generation.
It is easy to implement a moore machine using Circuit or program.	A mealy Machine is seldom implemented either using Circuit or program.

Q.7] Write a short note on NFA.

→ - An NFA is refered as Non deterministic finite automata.
- It is a theoretical model that accepts the steings.
- Unlike DFA, NFA can have multiple transitions for the same input symbol from a single state.
- It can have a transition on Empty state also.
- An NFA is defined by a 5-tuple
  $M = (Q, \Sigma, \delta, q_0, F)$
- The concept of non-deterministic finite automata is being explained with help of transition diagram



Q.8] Convert foll. grammar into finite Automata.
S = aX|bY|a|b , X = aS|bY|b , Y = aX|bS

→

S → aX|bY|a|b
X → aS|bY|b
Y → aX|bS

This is a right linear grammar which means it can be directly converted to grammar. FA.

Non terminals : S,X,Y
terminals : a,b

Convert productions into transitions

From S:
1. S → aX → δ(S,a) = X
2. S → bY → δ(S,b) = Y
3. S → a → δ(S,a) = F
4. S → b → δ(S,b) = F

From X:
1. X → aS → δ(X,a) = S
2. X → bY → δ(X,b) = Y
3. X → b → δ(X,b) = F

From Y:
1. Y → aX → δ(Y,a) = X
2. Y → bS → δ(Y,b) = S

	Current state	Input	Next state
1.	S	a	X,F
2.	S	b	Y,F
3.	X	a	S
4.	X	b	Y,F
5.	Y	a	X
6.	Y	b	S